Assignment-2

Software Tools and Techniques for CSE

Shardul Junagade (23110297)
Repository: cs202-stt

October 19, 2025

Contents

Lab Assignment 7: Reaching Definitions Analyzer for C Programs

Course: CS202 – Software Tools and Techniques for CSE
Lab Topic: Reaching Definitions Analyzer for C Programs
Name: Shardul Junagade
Roll Number: 23110297
Date: 15th September 2025

Repository Link(s): `lab7/` (runner: `lab7/run_analysis.py`, code: `lab7/analyzer/`)

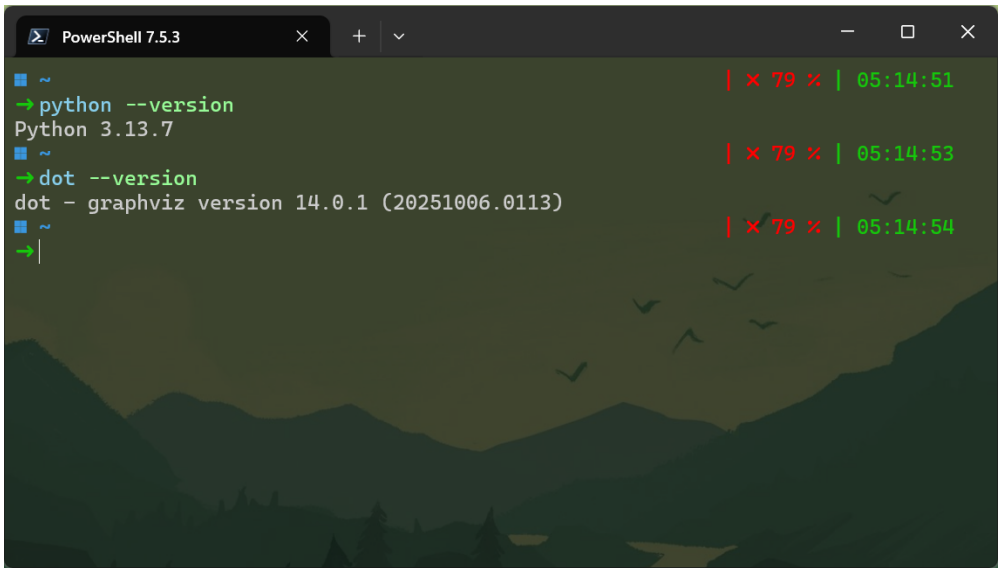
1.1 Introduction, Setup, and Tools

1.1.1 Introduction

I implemented an intraprocedural analyzer that constructed Control Flow Graphs (CFGs) for single-file C programs and ran Reaching Definitions analysis on top of them. I created DOT/PNG CFGs, per-iteration data-flow tables, a definitions index, an ambiguity report, and basic metrics including cyclomatic complexity.

1.1.2 Environment and Tools

- OS: Windows 11; Terminal: PowerShell 7
- Python: 3.13.7
- Required tools: Graphviz (for `dot`), Matplotlib (for plots)
- Editor: VS Code Insiders

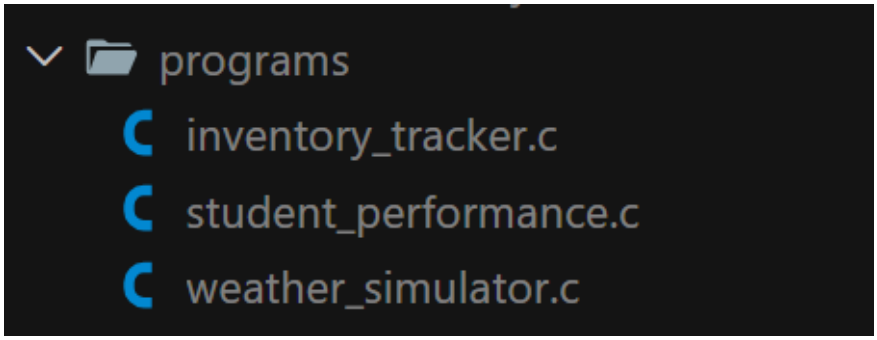


Environment Details

1.2 Program Corpus

I selected three single-file C programs (200–300 LOC each) that had conditionals, loops, and multiple reassignments, as required. I chose programs that were self-contained to avoid interprocedural complexity.

- Program 1: `inventory_tracker.c` – tracked stock levels with multiple conditionals and loop-based updates; good mix of assignments.
- Program 2: `student_performance.c` – computed aggregates with branch-heavy logic around grading thresholds.
- Program 3: `weather_simulator.c` – iterative simulation with nested condition checks and loop-carried updates.



`lab7/programs` folder snapshot

1.3 Approach and Implementation

1.3.1 Parsing and Cleaning – `analyzer/c_parser.py`

- I first read the source and stripped preprocessor lines and comments: Removed `#...` lines and both `/* ... */` and `// ...` comments.

```python
def read_c_file(path: str) -> str:
    with open(path, "r", encoding="utf-8") as handle:
        return handle.read()


def strip_preprocessor_and_comments(source: str) -> str:
    """Remove preprocessor lines and C/C++ style comments."""
    import re

    # Remove /* ... */ block comments
    source = re.sub(r"/\*.*?\*/", "", source, flags=re.S)
    # Remove // line comments
    source = re.sub(r"//.*", "", source)
    # Remove preprocessor lines
    lines = []
    for line in source.splitlines():
        if line.lstrip().startswith("#"):
            lines.append("")
        else:
            lines.append(line)
    return "\n".join(lines)
```

Code snippet – `strip_preprocessor_and_comments`

- Extracted only the body of `main()` and tracked its starting line so later line numbers stayed meaningful.

```python
def find_main_function_bounds(source: str) -> Tuple[int, int]:
    """Return (start_index, end_index) character offsets of main() body braces.

    start_index points to the first character after the opening '{'.
    end_index points to the closing '}' (exclusive).
    Raises ValueError if not found.
    """
    import re

    # Find 'main' function signature
    match = re.search(r"\bint\s+main\s*\([^)]*\)\s*\{", source)
    if not match:
        # Try without assuming return type strictly
        match = re.search(r"\bmain\s*\([^)]*\)\s*\{", source)
    if not match:
        raise ValueError("No function named 'main' found in the provided file.")
    open_brace_idx = source.find("{", match.end() - 1)
    if open_brace_idx == -1:
        raise ValueError("Malformed main(): missing opening '{'.")
    depth = 0
    i = open_brace_idx
    while i < len(source):
        ch = source[i]
        if ch == '{':
            depth += 1
        elif ch == '}':
            depth -= 1
            if depth == 0:
                # body is between open_brace_idx+1 and i (exclusive)
                return (open_brace_idx + 1, i)
        i += 1
    raise ValueError("Malformed main(): missing closing '}'.")


def extract_main_function_source(source: str) -> Tuple[str, int]:
    """Extract the body text of main() and its starting line number (1-based).

    Returns (body_text, start_line).
    """
    start, end = find_main_function_bounds(source)
    body = source[start:end]
    start_line = source[:start].count("\n") + 1
    return body, start_line
```

Code snippet – `extract_main_function_source`

- I kept it deliberately simple; this worked fine for my inputs but wasn't a full C parser.

1.3.2 Basic Blocks and CFG Construction – `analyzer/cfg_builder.py`

I used leader-based basic block detection per the instructions given in the lab document. Blocks were labeled `B0`, `B1`, ... and I added edges for sequential flow, conditionals, and simple loops:

- Leaders: first statement, the condition of `if/else if/else/while/for`, the statement after a branch/jump.

```python
# ---------------- Leader-based helpers -----------------
_COND_RE = re.compile(r'^\s*(if|else\s*if|else|while|for)\b')
_JUMP_RE = re.compile(r'^\s*(return|break|continue|goto)\b')

def _is_cond_or_loop(self, line: str) -> bool:
    return bool(self._COND_RE.match(line))

def _is_jump(self, line: str) -> bool:
    return bool(self._JUMP_RE.match(line))

def _find_leaders(self, lines: List[Tuple[str, int]]) -> List[int]:
    leaders: Set[int] = set()
    if lines:
        leaders.add(0)
    n = len(lines)
    for i, (txt, _) in enumerate(lines):
        if self._is_cond_or_loop(txt):
            leaders.add(i)
            if i + 1 < n:
                leaders.add(i + 1)
        if self._is_jump(txt):
            if i + 1 < n:
                leaders.add(i + 1)
        if txt.startswith('else'):
            leaders.add(i)
    return sorted(leaders)
```

Code snippet – `_find_leaders`

- Edges: for `if` I added true/false; for `while/for` I added back-edges from body to condition.
- Jumps like `return/break/continue` terminated flow for that block.

```python
def _build_edges_from_blocks(self, lines: List[Tuple[str, int]], blocks: List[Tuple[str, int, List[Tuple[str, int]]]]) -> List[Tuple[str, str, str]]:
    edges: List[Tuple[str, str, str]] = []
    n = len(blocks)
    for i, (bname, _start_idx, blines) in enumerate(blocks):
        if not blines:
            if i + 1 < n:
                edges.append((bname, f"B{i+1}", ""))
            continue
        first = blines[0][0]
        last = blines[-1][0]
        cond = self._is_cond_or_loop(first)
        jump = self._is_jump(last)
        if cond:
            if i + 1 < n:
                edges.append((bname, f"B{i+1}", "true"))
            if i + 2 < n:
                edges.append((bname, f"B{i+2}", "false"))
            if first.startswith('while') or first.startswith('for'):
                if i + 1 < n:
                    edges.append((f"B{i+1}", bname, "back"))
        elif not jump:
            if i + 1 < n:
                edges.append((bname, f"B{i+1}", ""))
    # de-duplicate
    seen: Set[Tuple[str, str, str]] = set()
    uniq: List[Tuple[str, str, str]] = []
    for e in edges:
        if e not in seen:
            uniq.append(e)
            seen.add(e)
    return uniq
```

Code snippet – `_build_edges_from_blocks`

I exported a DOT graph for each program, and rendered it to PNG if `dot` was in PATH using Graphviz. Following in the code snippet shows the export and rendering logic.

```python
def write_cfg_dot(cfg: CFG, destination: Path) -> None:
    lines: List[str] = ["digraph CFG {", "    node [shape=box];"]
    for block in cfg.blocks:
        label_lines = [f"{block.identifier}:"]
        for statement in block.statements:
            label_lines.append(statement.text)
        escaped = "\n".join(label_lines).replace("\"", "\\\"")
        lines.append(f"    {block.identifier} [label=\"{escaped}\"]; ")
    for block in cfg.blocks:
        for edge in block.successors:
            if edge.label:
                lines.append(
                    f"    {block.identifier} -> {edge.target} [label=\"{edge.label}\"];"
                )
            else:
                lines.append(f"    {block.identifier} -> {edge.target};")
    lines.append("}")
    destination.write_text("\n".join(lines), encoding="utf-8")


def render_dot_to_png(dot_file: Path, png_file: Optional[Path] = None) -> Optional[Path]:
    """Attempt to render a DOT file to PNG using Graphviz if available.

    Returns the path to the PNG file if successful, otherwise None.
    """
    if png_file is None:
        png_file = dot_file.with_suffix('.png')
    dot_executable = shutil.which("dot")
    if not dot_executable:
        print("Graphviz 'dot' executable not found in PATH. Skipping rendering.")
        return None
    try:
        subprocess.run([dot_executable, "-Tpng", str(dot_file), "-o", str(png_file)], check=True)
        return png_file
    except Exception:
        print(f"Failed to render {dot_file} to PNG.")
        return None
```

Code snippet – DOT export and PNG rendering

Link to sample CFG: inventory_tracker CFG PNG (too big to embed here)

1.3.3 Definition Extraction – `analyzer/cfg_builder.py`

For each statement, I extracted defined variables and created unique definition IDs (`D1`, `D2`, ...):

- Handled declarations like `int x = 0;`, `int x, y;` and simple assignments including compound ops like `+=`.
- Counted increments/decrements as definitions.
- Stored each definition with statement text, block ID, and line number.

I wrote a Markdown table (`definitions.md`) for quick reference. Following is the code snippet for the same:

```python
# -------------------- Definitions extraction --------------------
def _record_definitions_from_text(self, statement_text: str, line: int, block_id: str) -> List[str]:
    vars_defined = self._extract_defined_variables_from_text(statement_text)
    ids: List[str] = []
    for var in vars_defined:
        did = self._new_definition_id()
        self._definitions[did] = Definition(identifier=did, variable=var, statement=statement_text.strip(),
        line=line, block_id=block_id)
        self._var_to_defs.setdefault(var, set()).add(did)
        ids.append(did)
    return ids

def _extract_defined_variables_from_text(self, text: str) -> Set[str]:
    s = text.strip().rstrip(';')
    vars_defined: Set[str] = set()
    # handle declarations like 'int x = 0;' or 'int x, y;'
    decl_match = re.match(r"(int|float|double|char|long|short|unsigned|signed)\b(.*)", s)
    if decl_match:
        rest = decl_match.group(2)
        # split by commas outside parentheses
        parts = [p.strip() for p in rest.split(',')]
        for p in parts:
            name = re.split(r"\s|=|\[", p.strip())
            if name and name[0]:
                vars_defined.add(name[0].lstrip('*'))
    # handle assignments like 'x = ...' and increments 'x++' '--x'
    assign = re.match(r"([A-Za-z_][A-Za-z0-9_\.]*)\s*([+\-*/%&|^]?=)", s)
    if assign:
        vars_defined.add(assign.group(1))
    inc = re.match(r"([A-Za-z_][A-Za-z0-9_\.]*)\s*(\+\+|--)$", s)
    pre_inc = re.match(r"(\+\+|--)\s*([A-Za-z_][A-Za-z0-9_\.]*)$", s)
    if inc:
        vars_defined.add(inc.group(1))
    if pre_inc:
        vars_defined.add(pre_inc.group(2))
    return vars_defined
```

Code snippet – `_extract_defined_variables_from_text`

Definition ID	Variable	Block	Line	Statement
D1	categories	B0	2	`int categories = 8;`
D2	stores	B0	3	`int stores = 12;`
D3	weeks	B0	4	`int weeks = 10;`
D4	stock	B0	5	`int stock[8][12][10];`
D5	reorderThreshold	B0	6	`int reorderThreshold[8];`
D6	reorderCount	B0	7	`int reorderCount[8];`
D7	totalStock	B0	8	`int totalStock = 0;`
D8	totalDeficit	B0	9	`int totalDeficit = 0;`
D9	totalSurplus	B0	10	`int totalSurplus = 0;`
D10	outOfStock	B0	11	`int outOfStock = 0;`
D11	overStocked	B0	12	`int overStocked = 0;`
D12	balanced	B0	13	`int balanced = 0;`
D13	storeIndex	B0	14	`int storeIndex = 0;`
D14	categoryIndex	B0	15	`int categoryIndex = 0;`
D15	weekIndex	B0	16	`int weekIndex = 0;`
D16	forecast	B0	17	`int forecast[8][12];`
D17	demandSpike	B0	18	`int demandSpike[8][12];`
D18	demandDrop	B0	19	`int demandDrop[8][12];`
D19	movingAvg	B0	20	`int movingAvg[8][12];`
D20	trendGrowth	B0	21	`int trendGrowth = 0;`
D21	trendDecline	B0	22	`int trendDecline = 0;`
D22	trendStable	B0	23	`int trendStable = 0;`
D23	highestStock	B0	24	`int highestStock = 0;`
D24	lowestStock	B0	25	`int lowestStock = 0;`
D25	cumulativeVariance	B0	26	`int cumulativeVariance = 0;`
D26	maxVarianceCategory	B0	27	`int maxVarianceCategory = 0;`
D27	minVarianceCategory	B0	28	`int minVarianceCategory = 0;`
D28	tempVariance	B0	29	`int tempVariance = 0;`
D29	stockChange	B0	30	`int stockChange = 0;`
D30	previousStock	B0	31	`int previousStock = 0;`
D31	sameTrendCount	B0	32	`int sameTrendCount = 0;`

`definitions.md` preview

1.3.4 gen/kill Sets per Block – `analyzer/cfg_builder.py`

I computed `gen[B]` and `kill[B]` for each block:

- **gen[B] (generated definitions):** All definitions that are created inside block B.
- **kill[B] (killed definitions):** All other definitions of the same variables that exist outside block B.

I computed `gen[B]` and `kill[B]` as the latest definitions per variable within a block, in program order. The code snippet below shows the implementation (`compute_gen_kill_sets`).

```python
def _compute_gen_kill_sets(self) -> None:
    for block in self._blocks:
        latest_by_var: Dict[str, str] = {}
        new_gen: List[str] = []
        for statement in block.statements:
            for definition_id in statement.definition_ids:
                definition = self._definitions[definition_id]
                previous = latest_by_var.get(definition.variable)
                if previous and previous in new_gen:
                    new_gen.remove(previous)
                latest_by_var[definition.variable] = definition_id
                new_gen.append(definition_id)
        block.gen = new_gen
        kills: Set[str] = set()
        for variable, definition_id in latest_by_var.items():
            all_defs = self._var_to_defs.get(variable, set())
            kills.update(all_defs - {definition_id})
        block.kill = kills
        block.in_set = set()
        block.out_set = set(block.gen)
```

Code snippet – `_compute_gen_kill_sets`

1.3.5 Reaching Definitions (Data-Flow) – `analyzer/dataflow.py`

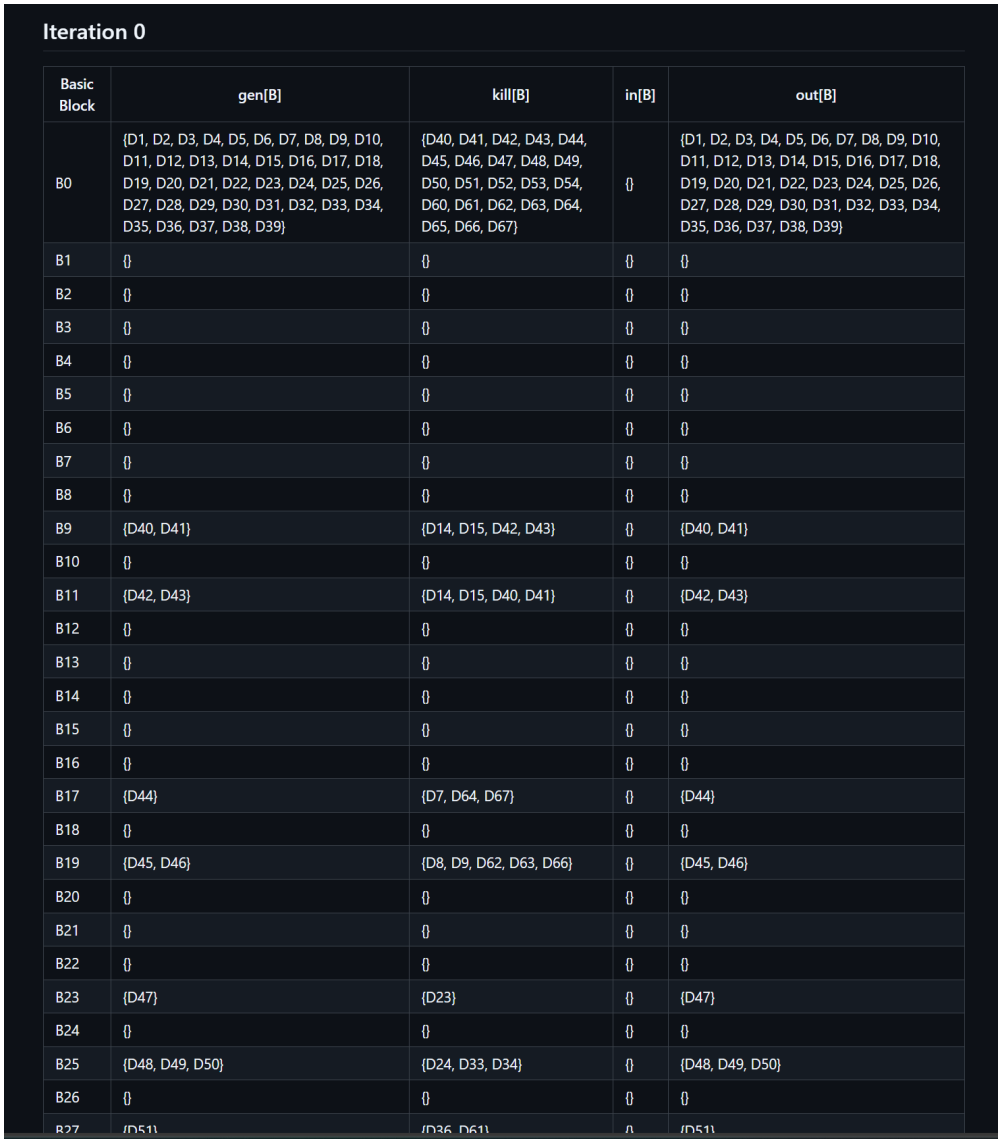
I implemented the standard forward data-flow equations (`analyzer/dataflow.py`):

- $in[B] = \bigcup out[P]$ over predecessors P
- $out[B] = gen[B] \cup (in[B] - kill[B])$

I iterated until the sets stopped changing. I recorded every iteration as a snapshot and wrote a Markdown table with `gen/kill/in/out` for each block per iteration. Following is the snippet for the iteration logic:

```python
def compute_reaching_definitions(cfg: CFG, max_iterations: int = 100) -> List[Dict[str, Dict[str, Set[str]]]]:
    block_map = cfg.block_map
    snapshots: List[Dict[str, Dict[str, Set[str]]]] = []
    initial_snapshot: Dict[str, Dict[str, Set[str]]] = {}
    for block in cfg.blocks:
        initial_snapshot[block.identifier] = {
            "gen": set(block.gen),
            "kill": set(block.kill),
            "in": set(block.in_set),
            "out": set(block.out_set),
        }
    snapshots.append(initial_snapshot)
    iteration = 0
    changed = True
    while changed and iteration < max_iterations:
        iteration += 1
        changed = False
        snapshot: Dict[str, Dict[str, Set[str]]] = {}
        for block in cfg.blocks:
            in_set: Set[str] = set()
            for predecessor_id in block.predecessors:
                predecessor = block_map[predecessor_id]
                in_set.update(predecessor.out_set)
            out_set = set(block.gen) | (in_set - block.kill)
            if in_set != block.in_set or out_set != block.out_set:
                block.in_set = in_set
                block.out_set = out_set
                changed = True
            snapshot[block.identifier] = {
                "gen": set(block.gen),
                "kill": set(block.kill),
                "in": set(block.in_set),
                "out": set(block.out_set),
            }
        snapshots.append(snapshot)
    return snapshots
```

Code snippet – `compute_reaching_definitions`



Iteration 0

Basic Block	gen[B]	kill[B]	in[B]	out[B]
B0	{D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D14, D15, D16, D17, D18, D19, D20, D21, D22, D23, D24, D25, D26, D27, D28, D29, D30, D31, D32, D33, D34, D35, D36, D37, D38, D39}	{D40, D41, D42, D43, D44, D45, D46, D47, D48, D49, D50, D51, D52, D53, D54, D60, D61, D62, D63, D64, D65, D66, D67}	{}	{D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D14, D15, D16, D17, D18, D19, D20, D21, D22, D23, D24, D25, D26, D27, D28, D29, D30, D31, D32, D33, D34, D35, D36, D37, D38, D39}
B1	{}	{}	{}	{}
B2	{}	{}	{}	{}
B3	{}	{}	{}	{}
B4	{}	{}	{}	{}
B5	{}	{}	{}	{}
B6	{}	{}	{}	{}
B7	{}	{}	{}	{}
B8	{}	{}	{}	{}
B9	{D40, D41}	{D14, D15, D42, D43}	{}	{D40, D41}
B10	{}	{}	{}	{}
B11	{D42, D43}	{D14, D15, D40, D41}	{}	{D42, D43}
B12	{}	{}	{}	{}
B13	{}	{}	{}	{}
B14	{}	{}	{}	{}
B15	{}	{}	{}	{}
B16	{}	{}	{}	{}
B17	{D44}	{D7, D64, D67}	{}	{D44}
B18	{}	{}	{}	{}
B19	{D45, D46}	{D8, D9, D62, D63, D66}	{}	{D45, D46}
B20	{}	{}	{}	{}
B21	{}	{}	{}	{}
B22	{}	{}	{}	{}
B23	{D47}	{D23}	{}	{D47}
B24	{}	{}	{}	{}
B25	{D48, D49, D50}	{D24, D33, D34}	{}	{D48, D49, D50}
B26	{}	{}	{}	{}
B27	{D51}	{D36, D61}	{}	{D51}

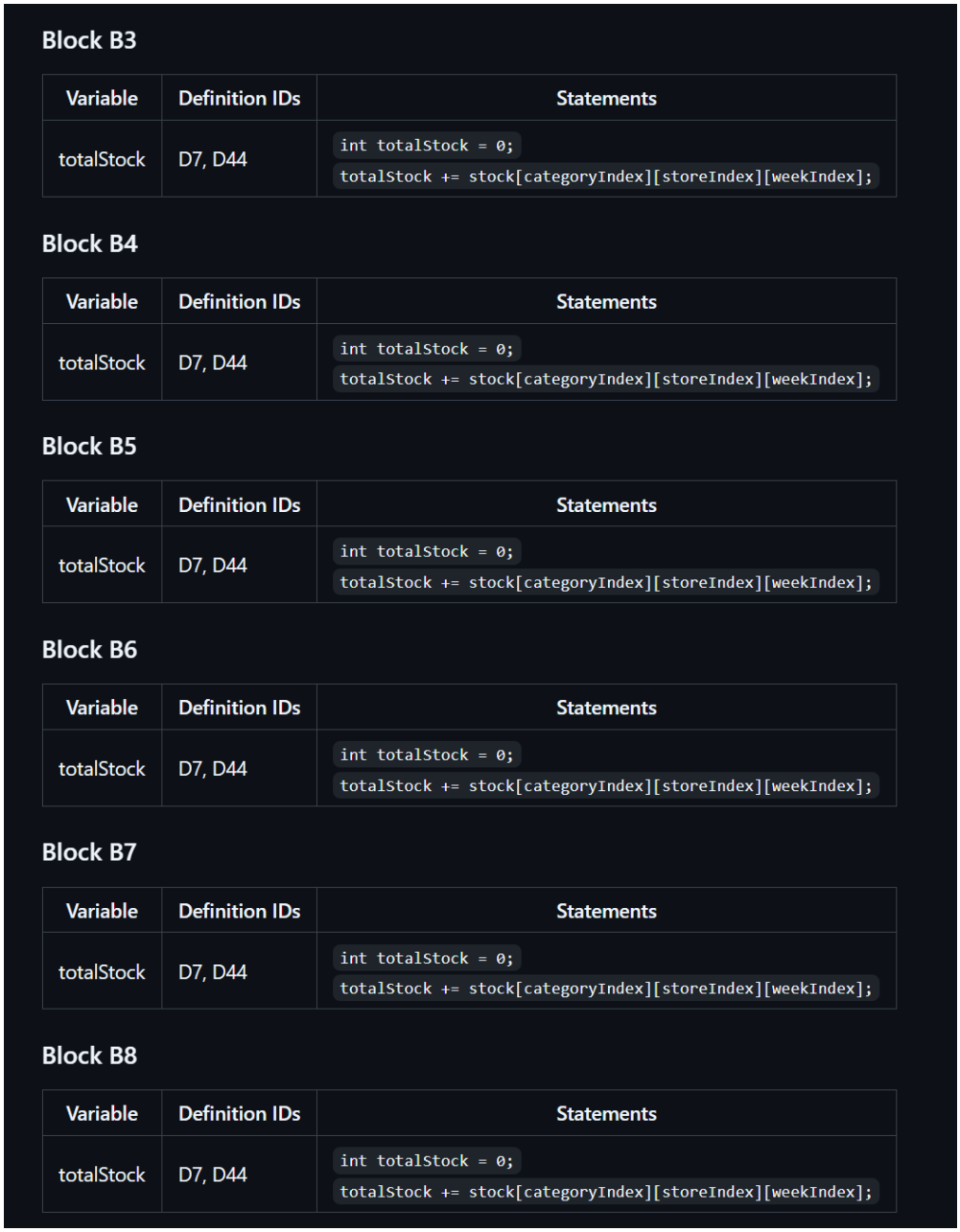
`reaching_definitions_iterations.md` preview

1.3.6 Ambiguity Detection – `analyzer/dataflow.py`

After convergence, I flagged blocks where a variable had more than one reaching definition in `in[B]` (`find_ambiguous_variables`). I summarized them with the corresponding statements for context (`ambiguity.md`).

```python
def find_ambiguous_variables(cfg: CFG) -> Dict[str, Dict[str, Set[str]]]:
    ambiguous: Dict[str, Dict[str, Set[str]]] = {}
    for block in cfg.blocks:
        var_to_defs: Dict[str, Set[str]] = {}
        for definition_id in block.in_set:
            definition = cfg.definitions[definition_id]
            var_to_defs.setdefault(definition.variable, set()).add(definition_id)
        ambiguous_vars = {var: defs for var, defs in var_to_defs.items() if len(defs) > 1}
        if ambiguous_vars:
            ambiguous[block.identifier] = ambiguous_vars
    return ambiguous
```

Code snippet – `find_ambiguous_variables`



Block B3

Variable	Definition IDs	Statements
totalStock	D7, D44	`int totalStock = 0;` `totalStock += stock[categoryIndex][storeIndex][weekIndex];`

Block B4

Variable	Definition IDs	Statements
totalStock	D7, D44	`int totalStock = 0;` `totalStock += stock[categoryIndex][storeIndex][weekIndex];`

Block B5

Variable	Definition IDs	Statements
totalStock	D7, D44	`int totalStock = 0;` `totalStock += stock[categoryIndex][storeIndex][weekIndex];`

Block B6

Variable	Definition IDs	Statements
totalStock	D7, D44	`int totalStock = 0;` `totalStock += stock[categoryIndex][storeIndex][weekIndex];`

Block B7

Variable	Definition IDs	Statements
totalStock	D7, D44	`int totalStock = 0;` `totalStock += stock[categoryIndex][storeIndex][weekIndex];`

Block B8

Variable	Definition IDs	Statements
totalStock	D7, D44	`int totalStock = 0;` `totalStock += stock[categoryIndex][storeIndex][weekIndex];`

`ambiguity.md` preview

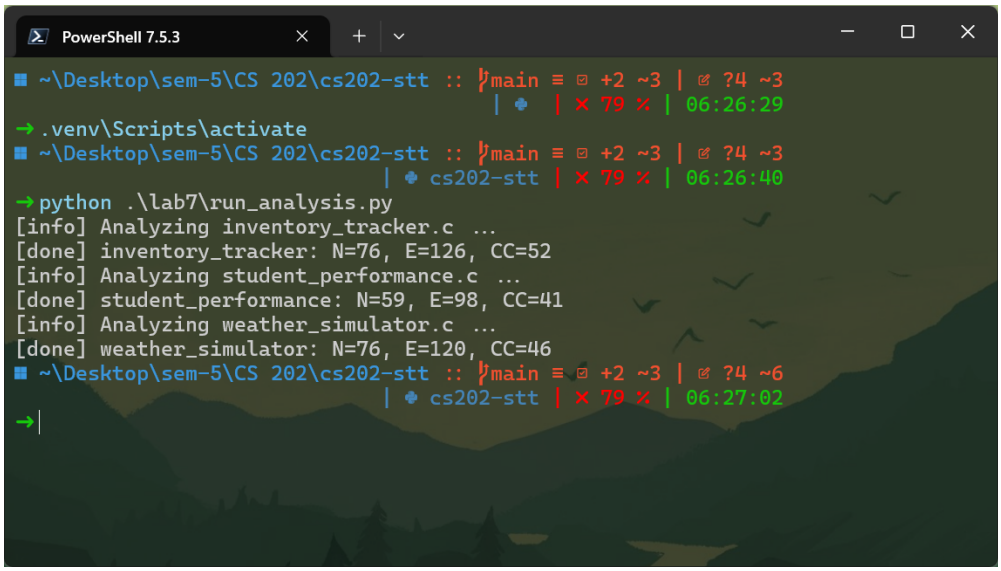
1.3.7 Runner and Outputs

The runner (`lab7/run_analysis.py`) processed all `.c` files in `lab7/programs/` and wrote per-program outputs to `lab7/output/<program_name>/`:

- `cfg.dot` (+ `cfg.png` if Graphviz was available)
- `definitions.md`
- `reaching_definitions_iterations.md`
- `ambiguity.md`
- `summary.json` (nodes, edges, CC)

```python
def analyze_program(program_path: Path) -> Dict[str, int]:
    print(f"[info] Analyzing {program_path.name} ...")
    source = read_c_file(str(program_path))
    cleaned = strip_preprocessor_and_comments(source)
    body, start_line = extract_main_function_source(cleaned)

    cfg = build_cfg_from_main_source(body, start_line)

    snapshots = compute_reaching_definitions(cfg)
    ambiguous = find_ambiguous_variables(cfg)

    nodes = len(cfg.blocks)
    edges = sum(len(block.successors) for block in cfg.blocks)
    cyclomatic_complexity = edges - nodes + 2

    program_name = program_path.stem
    program_output_dir = OUTPUT_DIR / program_name
    program_output_dir.mkdir(parents=True, exist_ok=True)

    dot_path = program_output_dir / "cfg.dot"
    write_cfg_dot(cfg, dot_path)
    render_dot_to_png(dot_path)
    write_definitions_table(cfg.definitions, program_output_dir / "definitions.md")
    write_iteration_tables(snapshots, cfg, program_output_dir / "reaching_definitions_iterations.md")
    write_ambiguity_report(ambiguous, cfg.definitions, program_output_dir / "ambiguity.md")
    write_program_summary(
        program_name,
        {"nodes": nodes, "edges": edges, "cc": cyclomatic_complexity},
        program_output_dir / "summary.json",
    )

    print(f"[done] {program_name}: N={nodes}, E={edges}, CC={cyclomatic_complexity}")
    return {"program": program_name, "nodes": nodes, "edges": edges, "cc": cyclomatic_complexity}
```

Code snippet – Runner invoking analysis and writing outputs



Terminal Output

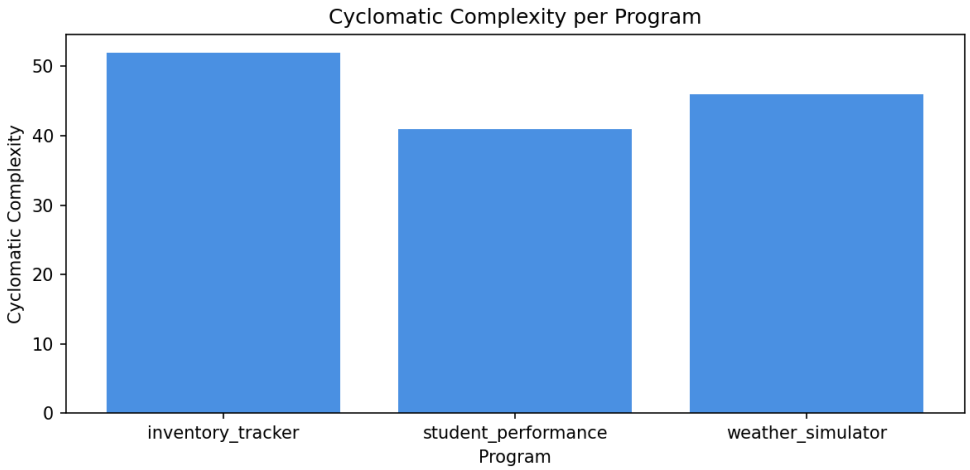
I also wrote code to generate an overall `metrics_summary.md` and, if Matplotlib was available, a bar chart `cyclomatic_complexity.png` at `lab7/output/`.

```python
def generate_metrics_summary(metrics: List[Dict[str, int]]) -> None:
    rows = [
        "| Program | Nodes (N) | Edges (E) | Cyclomatic Complexity (CC) |",
        "|---------|-----------|-----------|----------------------------|",
    ]
    for entry in metrics:
        rows.append(
            f"| {entry['program']} | {entry['nodes']} | {entry['edges']} | {entry['cc']} |"
        )
    (OUTPUT_DIR / "metrics_summary.md").write_text("\n".join(rows), encoding="utf-8")
    plot_cyclomatic_complexity(metrics, OUTPUT_DIR / "cyclomatic_complexity.png")
```

Code snippet – Metrics summary generation

Program	Nodes (N)	Edges (E)	Cyclomatic Complexity (CC)
inventory_tracker	76	126	52
student_performance	59	98	41
weather_simulator	76	120	46

`metrics_summary.md` preview



Cyclomatic Complexity Chart

1.4 Results

I am attaching links to key outputs for each program below because they are too large to embed here.

1.4.1 CFG and Metrics

Links to CFG PNGs for each program:

- inventory_tracker
- student_performance
- weather_simulator

For each program, I reported the number of nodes (basic blocks), edges, and cyclomatic complexity `CC = E - N + 2`. The chart helped me compare CCs across the three programs at a glance.

Actual metrics from my run (`lab7/output/metrics_summary.md`):

Program	Nodes (N)	Edges (E)	Cyclomatic Complexity (CC)
inventory_tracker	76	126	52
student_performance	59	98	41
weather_simulator	76	120	46

1.4.2 Reaching Definitions Tables

I skimmed early iterations to sanity-check propagation, and I confirmed convergence when tables stopped changing. I then used the final snapshot to interpret variable reachability.

Links to final iteration tables for each program:

- inventory_tracker
- student_performance
- weather_simulator

1.4.3 Ambiguity Highlights

I listed a few examples where variables had multiple reaching definitions at a program point and explained why (e.g., different branches or loop-carried definitions).

Links to detailed ambiguity tables for each program:

- inventory_tracker
- student_performance
- weather_simulator

1.5 Discussion and Reflection

1.5.1 Challenges

I faced a few challenges while implementing the analyzer. Getting the leader-based heuristics to behave well across nested conditionals, chained `else if` constructs, and loops required careful compromises. I deliberately kept the reconstruction simple, which worked for my programs but isn't a full CFG reconstructor. I also had to filter out syntactic noise early (like standalone braces and empty statements) to keep basic blocks clean. Finally, I ensured that Graphviz was installed and on PATH by updating the environment variables on my Windows machine.

1.5.2 What I Learned

This assignment gave me a hands-on understanding of how control-flow and data-flow analyses fit together in practice. Building CFGs forced me to think in terms of basic blocks, leaders, and edges for branches and loops. I got a better understanding of data-flow analysis by defining `gen/kill` precisely and then iterating the reaching definitions equations $in[B] = \bigcup out[P]$ and $out[B] = gen[B] \cup (in[B] - kill[B])$. Tracing iteration snapshots helped me verify convergence and understand why certain variables had multiple reaching definitions at a point (ambiguity due to branches or loop-carried updates). Overall, I learned how to move from raw C source to CFGs, to `gen/kill` sets, to stable reaching definitions.

1.5.3 Further Improvements

- Swap the lightweight parsing for a proper C front-end (e.g., `pycparser`) to cover more syntax and nesting reliably.
- Refine CFG edges for more complex constructs and labels.
- Add unit tests for tricky statements (declarations with pointers/arrays, compound statements).

1.6 References

- Lecture 7 slides: here
- Data-flow analysis: `https://en.wikipedia.org/wiki/Data-flow_analysis`
- Graphviz: `https://graphviz.org/`
- PyGraphviz (optional): `https://pygraphviz.github.io/`